

Rim Defect Detection

The Complete Mathematics Reference

Every formula, equation, and mathematical operation used across
computer-vision-notebook_v4_0.ipynb and tpu-notebook-computervision.ipynb,
written out clearly with full explanation.

Updated: May 2026

Table of Contents

Section 1	— Image Normalisation
Section 2	— Data Augmentation Mathematics
Section 3	— Oversampling & Class Weighting
Section 4	— Attention Mechanisms — CBAM
Section 5	— Attention Pooling (AttentionPool2d)
Section 6	— Cross-Backbone Multi-Head Attention
Section 7	— Activation Functions (Sigmoid, GELU, ReLU, Softmax)
Section 8	— Loss Functions
Section 9	— Online Hard Example Mining (OHEM)
Section 10	— Exponential Moving Average (EMA)
Section 11	— MixUp & CutMix
Section 12	— Test-Time Augmentation (TTA) — D4 Group
Section 13	— Optimiser — AdamW
Section 14	— Learning Rate Schedule — Cosine Annealing
Section 15	— Gradient Clipping
Section 16	— Gradient Accumulation & Effective Batch Size
Section 17	— Mixed Precision & Loss Scaling (GPU Only)
Section 18	— Batch Normalisation & Recalibration
Section 19	— Stochastic Weight Averaging (SWA)
Section 20	— Platt Scaling (Calibration)
Section 21	— Evaluation Metrics
Section 22	— Threshold Sweep
Section 23	— Cascade Inference Decision Rule
Section 24	— Rim Cropping — HoughCircles
Section 25	— Grad-CAM (v4.0 GPU only)
Section 26	— Pseudo-Labelling Confidence Filter
Section 27	— Vignette, Ring & Specular Highlight Geometry

1. Image Normalisation

Before any image enters the neural network, each pixel value is normalised to match the statistics of the ImageNet dataset that the pretrained backbones were trained on. This ensures the model's learned weights remain meaningful.

Per-channel normalisation formula

$$x_norm[c] = (x[c] / 255 - mean[c]) / std[c]$$

where c is the colour channel (R, G, B), $x[c]$ is the raw pixel value 0–255, and the constants are:

$$mean = [0.485, 0.456, 0.406] \text{ (R, G, B)} \quad std = [0.229, 0.224, 0.225] \text{ (R, G, B)}$$

These are the empirical mean and standard deviation of all pixel values across the ImageNet-1k training set (1.2 million images). After normalisation each channel has approximately mean 0 and variance 1, which stabilises early gradient flow through the pretrained weights.

2. Data Augmentation Mathematics

2a. Specular Highlight — 2D Gaussian Ellipse

A Gaussian-shaped bright patch is added to simulate metallic glare from factory lighting. For each pixel (x, y) the mask intensity is:

```
mask(x, y) = exp( -[ ((x - cx)/rx)^2 + ((y - cy)/ry)^2 ] ) pixel_out[c] =  
clip( pixel_in[c] + mask(x,y) * intensity * 255 , 0, 255 )
```

cx, cy is the random ellipse centre; rx, ry are random semi-axes; intensity ~ Uniform(0.2, 0.7). The Gaussian falls off smoothly from the centre, replicating the Lambertian + specular reflection model of shiny metal surfaces.

2b. SectorMask — Pie Slice Occlusion

A circular sector (pie slice) of the image is filled with the image's mean colour. The bounding radius is the image's circumscribed circle:

```
r_bound = ceil( sqrt( (W/2)^2 + (H/2)^2 ) ) + 1 sector = ellipse centred at  
(W/2, H/2) with radius r_bound, swept from start_angle to start_angle +  
sweep_angle start_angle ~ Uniform(0, 360 deg) sweep_angle ~ Uniform(10,  
max_angle deg) where max_angle = 60
```

2c. Vignette — Normalised Radial Distance

Corners are darkened by multiplying each pixel by a vignette factor computed from its normalised distance from the image centre:

```
dist(x,y) = sqrt( ((x - W/2)/(W/2))^2 + ((y - H/2)/(H/2))^2 ) vignette(x,y)  
= 1 - clip( dist(x,y) * strength , 0 , 0.6 ) pixel_out = pixel_in *  
vignette(x,y) strength ~ Uniform(0.3, 0.7)
```

2d. Ring Light — Annular Mask

A faint ring is added by computing Euclidean distance from the image centre and selecting pixels within an annulus [r1, r2]:

```
dist(x,y) = sqrt( (x - W/2)^2 + (y - H/2)^2 ) ring(x,y) = 1 if r1 <=  
dist(x,y) <= r2, else 0 pixel_out = pixel_in + ring(x,y) * tint * 30 r1 ~  
Uniform(0.2, 0.4) * min(W,H) r2 = r1 + Uniform(5, 20)
```

2e. ScratchSimulation — Trigonometric Line Placement

Random thin lines are drawn at random angles. End-point coordinates are computed from a start point, length, and angle:

```
x2 = int( x1 + length * cos(angle) ) y2 = int( y1 + length * sin(angle) )  
length ~ Uniform(20, W/2) angle ~ Uniform(0, pi)
```

3. Oversampling & Class Weighting

3a. Oversampling — Minority Duplication

Let n_0 = number of majority-class samples, n_1 = minority-class samples ($n_1 < n_0$). The number of extra duplicates needed to reach a target ratio $r = 1.0$ is:

```
n_extra = floor( n0 * r ) - n1
The minority set is sampled WITH replacement
n_extra times: extra[i] ~ UniformChoice(minority_samples)
```

After oversampling: total minority = $n_1 + n_extra = n_0$, so both classes contribute equally to each training epoch.

3b. WeightedRandomSampler — Per-Sample Probability

When oversampling is disabled, a `WeightedRandomSampler` assigns each sample a draw probability inversely proportional to its class frequency:

```
w0 = 1 / n0 (weight for majority class) w1 = 1 / n1 (weight for minority
class) P(sample i drawn) = w[label_i] / sum_j w[label_j]
```

3c. Positive Weight for BCEWithLogitsLoss

When the dataset is NOT oversampled, a `pos_weight` term rescales the loss contribution of positive (defective) samples:

```
pos_weight = n0 / n1 BCE_weighted(logit, y) = -[ pos_weight * y *
log(sigma(logit)) + (1-y) * log(1 - sigma(logit)) ]
When oversampling IS active: pos_weight = 1.0 (balanced dataset)
```

4. Attention Mechanisms — CBAM

CBAM (Convolutional Block Attention Module) applies two sequential attention gates to a feature map X of shape (B, C, H, W) .

4a. Channel Attention Gate

Both global average pooling and global max pooling compress spatial dimensions. Their results are passed through a shared 2-layer MLP (reduction ratio $r = 16$) and summed before sigmoid gating:

```
F_avg = AvgPool(X) shape: (B, C, 1, 1) F_max = MaxPool(X) shape: (B, C, 1, 1)
MLP(v) = W2 * ReLU( W1 * v ) W1: (C, C/r) W2: (C/r, C) M_c = sigma(
MLP(F_avg.squeeze()) + MLP(F_max.squeeze()) ) shape: (B, C) X' = X *
M_c.unsqueeze(-1).unsqueeze(-1)
```

M_c is the channel attention map — a scalar per channel indicating how important that feature channel is for the current image.

4b. Spatial Attention Gate

After channel attention, a spatial map is computed by concatenating channel-wise average and max projections, then applying a 7×7 convolution:

```
sp_in = cat( mean(X', dim=C), max(X', dim=C) ) shape: (B, 2, H, W) M_s =
sigma( Conv7x7(sp_in) ) shape: (B, 1, H, W) X'' = X' * M_s
```

M_s is the spatial attention map — a per-pixel importance score. The final output X'' focuses both on the right channels and the right pixels.

5. Attention Pooling (AttentionPool2d)

Standard Global Average Pooling (GAP) computes an unweighted mean over all spatial positions. For anomaly detection this is destructive — a crack covering 1% of the image is averaged with 99% of clean metal.

GAP (baseline, NOT used for defect features)

```
GAP(X) = (1 / H*W) * sum_{i,j} X[:, :, i, j] shape: (B, C)
```

Soft-attention pooling (used in the ensemble)

A 1×1 convolution scores every spatial position. Softmax converts scores to a probability distribution, then a weighted sum replaces the flat average:

```
scores = Conv1x1(X) shape: (B, 1, H, W) w = softmax( scores.flatten(dim=2)
) shape: (B, 1, H*W) AttPool(X) = sum_k w[:, 0, k] * X_flat[:, :, k] shape:
(B, C) where X_flat = X.flatten(2) shape: (B, C, H*W)
```

The weights w are learned end-to-end. The model learns to assign high weight to positions containing defect signals and near-zero weight to clean background areas.

6. Cross-Backbone Multi-Head Attention

After each backbone produces a feature vector (shape $B \times C_i$), they are projected to a common embedding dimension $d = 256$ and stacked into a sequence of 3 tokens. Multi-Head Self-Attention then lets each backbone attend to the other two.

6a. Linear Projection

```
f1' = W1 * f1 (B, C1) -> (B, 256) f2' = W2 * f2 (B, C2) -> (B, 256) f3' = W3 * f3 (B, C3) -> (B, 256) seq = stack([f1', f2', f3'], dim=1) shape: (B, 3, 256)
```

6b. Scaled Dot-Product Attention (per head)

With $h = 8$ heads, $d_{\text{head}} = 256 / 8 = 32$. For each head k :

```
Q_k = seq * W_Q_k K_k = seq * W_K_k V_k = seq * W_V_k head_k = softmax( Q_k * K_k^T / sqrt(d_head) ) * V_k MultiHead = concat(head_1, ..., head_8) * W_0 shape: (B, 3, 256)
```

The division by $\sqrt{d_{\text{head}}} = \sqrt{32} \approx 5.66$ prevents the dot products from growing too large and pushing softmax into saturation.

6c. Fusion — Mean Pooling + LayerNorm

```
attn_out = MultiHead(seq, seq, seq) shape: (B, 3, 256) fused = LayerNorm(mean(attn_out, dim=1) ) shape: (B, 256) LayerNorm(x) = (x - mean(x)) / sqrt(var(x) + 1e-5) * gamma + beta
```

7. Activation Functions

7a. Sigmoid

$\text{sigma}(x) = 1 / (1 + \exp(-x))$ range: (0, 1)

Used to convert raw logits to probabilities for binary classification. Also used inside CBAM attention gates.

7b. GELU (Gaussian Error Linear Unit)

$\text{GELU}(x) = x * \Phi(x)$ where $\Phi(x) = P(X \leq x)$ for $X \sim N(0,1)$ [standard normal CDF] Approximation: $\text{GELU}(x) \approx 0.5 * x * (1 + \tanh(\sqrt{2/\pi} * (x + 0.044715 * x^3)))$

Used in the classifier head and the cross-attention linear layers. GELU is smoother than ReLU near zero and is preferred in Transformer-style architectures.

7c. ReLU (Rectified Linear Unit)

$\text{ReLU}(x) = \max(0, x)$

Used inside the CBAM channel-attention MLP and inside Grad-CAM (applied to the raw CAM to remove negative activations).

7d. Softmax

$\text{softmax}(x)_i = \exp(x_i) / \sum_j \exp(x_j)$ Used in: attention weights (CBAM spatial, AttentionPool2d, Multi-Head Attention)

8. Loss Functions

8a. Binary Cross-Entropy (BCE) — base formula

```
BCE(logit, y) = -[ y * log(sigma(logit)) + (1-y) * log(1 - sigma(logit)) ]
```

8b. Label Smoothing

Before computing BCE, hard labels {0, 1} are softened by epsilon = 0.02 (v4.0) or 0.01 (TPU) to prevent the model from becoming overconfident:

```
y_smooth = y * (1 - eps) + 0.5 * eps Example (eps=0.02): y=1 ->
y_smooth=0.99 y=0 -> y_smooth=0.01
```

8c. Focal Loss

Focal Loss down-weights easy examples (high confidence, correct) and focuses the model on hard examples. Let $p = \text{sigma}(\text{logit})$:

```
pt = p * y + (1-p) * (1-y) 'probability of true class' FocalLoss(logit, y)
= (1 - pt)^gamma * BCE(logit, y) gamma = 2.5 (TPU) or 2.0 (GPU v4.0) When
pt -> 1 (easy correct): (1-pt)^2 -> 0 loss suppressed When pt -> 0 (hard
wrong): (1-pt)^2 -> 1 loss preserved
```

With pos_weight w applied only to the positive ($y=1$) term, the weighted BCE becomes: $-[w*y*\log(\text{sigma}(z)) + (1-y)*\log(1-\text{sigma}(z))]$.

8d. Asymmetric Loss (ASL)

ASL uses different focusing exponents for positive and negative samples. Let $p = \text{sigma}(\text{logit})$, $p_{\text{clip}} = \text{clip}(p + \text{delta}, \text{max}=1-\text{eps})$ with $\text{delta} = 0.05$:

```
L_pos = -y * log(clip(p, min=eps)) L_neg = -(1-y) * log(1 - p_clip) pt = p
* y + (1-p) * (1-y) w_asl = (1-pt)^gamma_pos if y>=0.5 (1-pt)^gamma_neg if
y<0.5 gamma_pos = 1 gamma_neg = 4 ASL(logit, y) = -(L_pos + L_neg) * w_asl
```

$\text{gamma_neg}=4$ aggressively suppresses easy negatives (clean rims the model already classifies correctly), so training energy goes to the hard cases.

8e. CombinedLoss — the full training objective

```
L_bin = (1 - w_asl) * Focal(z_bin, y) + w_asl * ASL(z_bin, y) where w_asl =
0.40 L_sev = MSE( sigma(z_sev), y ) = (sigma(z_sev) - y)^2 L_total = L_bin
+ w_sev * L_sev where w_sev = 0.25 Expanded: L_total = 0.60 * Focal + 0.40
* ASL + 0.25 * MSE_severity
```

The severity MSE term acts as a regulariser sharing gradients with the binary head. If the binary logit says 'defective', the severity logit should also be high, otherwise L_{sev} penalises the discrepancy.

9. Online Hard Example Mining (OHEM)

OHEM computes the loss for each sample in a batch individually, sorts them, and backpropagates only through the top-k hardest samples.

```
losses = [ L_total(z_bin_i, z_sev_i, y_i) for i in 0..B-1 ] k = max(1,
floor(B * keep_frac)) keep_frac = 0.70 hard_losses = top_k(losses, k,
largest=True) L_OHEM = mean(hard_losses)
```

With keep_frac=0.70 and batch size B=24, k=16. The 8 easiest images per batch are dropped from the gradient update. Over time this forces the model to focus on the ambiguous micro-scratches it keeps getting wrong.

10. Exponential Moving Average (EMA)

Update rule per training step t

```
d_t = min( decay, (1 + t) / (10 + t) ) warmup ramp shadow[t+1] = d_t *  
shadow[t] + (1 - d_t) * theta[t] decay = 0.9998 (GPU v4.0) or 0.9999 (TPU)
```

At step $t=0$: $d = \min(0.9998, 1/10) = 0.1$ — shadow moves quickly toward the model. At step $t=1000$: $d \approx 0.9998$ — shadow moves very slowly, averaging over many recent model states.

Effective averaging window

```
Effective window  $\approx 1 / (1 - \text{decay})$  GPU:  $1 / (1 - 0.9998) = 5000$  steps TPU:  
 $1 / (1 - 0.9999) = 10000$  steps
```

The EMA model's parameters are a soft average of the last ~5000 gradient updates (GPU) or ~10000 (TPU). This filters out noisy individual gradient steps and generalises better than any single checkpoint.

11. MixUp & CutMix

Shared: Beta distribution sampling

```
lambda ~ Beta(alpha, alpha) alpha = 0.20 (both notebooks) Beta(alpha, alpha) is symmetric; mode at 0.5 when alpha<1, concentrated near 0.5 when alpha>1.
```

MixUp (50% probability when `_mixup_cutmix` is called)

```
idx = random permutation of {0, ..., B-1} x_mix = lambda * x + (1-lambda) * x[idx] y_mix = lambda * y + (1-lambda) * y[idx]
```

The image is a pixel-level blend of two images. The label is a convex combination of the two labels (soft target).

CutMix (50% probability)

```
cut_r = sqrt(1 - lambda) fraction of side to cut cut_h = floor(H * cut_r) cut_w = floor(W * cut_r) cx ~ Uniform(0, W) cy ~ Uniform(0, H) x1 = max(0, cx - cut_w/2) x2 = min(W, cx + cut_w/2) y1 = max(0, cy - cut_h/2) y2 = min(H, cy + cut_h/2) x_mix[:, :, y1:y2, x1:x2] = x[idx, :, :, y1:y2, x1:x2] lambda_actual = 1 - (x2-x1)*(y2-y1) / (H*W) 'true area ratio' y_mix = lambda_actual * y + (1-lambda_actual) * y[idx]
```

The label uses `lambda_actual` (the true pixel fraction) not `lambda` (the Beta sample) because the clip to image boundaries changes the actual area.

12. Test-Time Augmentation — D4 Dihedral Group

A square image has exactly 8 unique symmetries forming the dihedral group D4. TTA applies all 8 transforms, gets 8 predictions, and averages:

```
D4 elements: e identity r1 rotate 90 deg clockwise r2 rotate 180 deg r3
rotate 270 deg fH flip horizontal fH o r1 flip horizontal then rotate 90 fV
flip vertical fV o r1 flip vertical then rotate 90 p_TTA = (1/8) * sum_{g
in D4} sigma( model( g(x) ) ) Final prediction: y_hat = 1 if p_TTA >=
threshold, else 0
```

Averaging 8 predictions reduces variance — if the model is uncertain about a scratch's orientation, some views will show it more clearly and the average is more robust than any single prediction.

13. Optimiser — AdamW

AdamW is used for all trainable parameters in both notebooks. At each gradient step t for parameter θ :

```
g_t = gradient of L w.r.t. theta
m_t = beta1 * m_{t-1} + (1-beta1) * g_t
'first moment'
v_t = beta2 * v_{t-1} + (1-beta2) * g_t^2
'second moment'
m_hat = m_t / (1 - beta1^t)
'bias-corrected'
v_hat = v_t / (1 - beta2^t)
theta_{t+1} = theta_t - lr * m_hat / (sqrt(v_hat) + eps) - lr *
weight_decay * theta_t
'decoupled WD'
Defaults: beta1=0.9 beta2=0.999
eps=1e-8 lr = 5e-5 weight_decay = 1e-4
```

The 'W' in AdamW refers to decoupled weight decay. In standard Adam, L2 regularisation is applied through the gradient (coupled). AdamW applies it directly to the parameter update (decoupled), which is more effective when using adaptive learning rates.

14. Learning Rate Schedule — Cosine Annealing

```
lr_t = lr_min + 0.5 * (lr_max - lr_min) * (1 + cos( pi * t / T_max )) t =  
current epoch within phase T_max = total epochs in phase (5, 8, or 27)  
lr_max = base_lr = 5e-5 lr_min = lr_max * 0.01 = 5e-7
```

At $t=0$: $lr = lr_{max}$. At $t=T_{max}$: $lr = lr_{min}$. The cosine curve gives a smooth warm start and slow cool-down, avoiding abrupt learning rate changes that destabilise training.

Different parameter groups get different base learning rates in Phase 3 (full fine-tuning). The backbone parameters use a lower multiplier (0.02–0.04×) to preserve pretrained features, while the classification head uses the full `base_lr`.

15. Gradient Clipping

```
g_norm = sqrt( sum_i ||grad_i||^2 ) global L2 norm if g_norm > max_norm:  
grad_i <- grad_i * (max_norm / g_norm) for all i max_norm = 0.5 (both  
notebooks)
```

Gradient clipping rescales all parameter gradients by the same factor when their global L2 norm exceeds 0.5. This prevents gradient explosions — sudden massive weight updates that can permanently destroy learned features.

16. Gradient Accumulation & Effective Batch Size

Gradient accumulation simulates a larger batch size by summing gradients over multiple forward-backward passes before an optimiser step.

```
loss_step = L_total(batch) / accum_steps for step in range(accum_steps):  
    loss_step.backward() # accumulates into .grad buffers optimizer.step() #  
    one weight update optimizer.zero_grad() effective_batch_size = batch_size *  
    accum_steps GPU: 2 * 32 = 64 TPU: 192 * 1 = 192 (real batch, no  
    accumulation needed)
```

Dividing the loss by `accum_steps` before `backward()` keeps the gradient magnitude the same as a single forward pass on the full effective batch. Without this division, gradients would be `accum_steps` times too large.

17. Mixed Precision & Loss Scaling (GPU Only)

On Volta/Turing GPUs (V100, T4), float16 is used for the forward pass to halve memory usage and speed up computation. However float16 can represent values only down to approximately 6×10^{-5} . Gradients smaller than this underflow to zero.

Loss scaling (GradScaler) — GPU v4.0 with float16

```
scaled_loss = loss * scale_factor  # scale_factor typically 2^16
scaled_loss.backward()  # scaled gradients, no underflow
scaler.unscale_(optimizer)  # divide all grads by scale_factor
clip_grad_norm_(model.parameters(), max_norm)
scaler.step(optimizer)
scaler.update()  # adapts scale_factor each step
```

bfloat16 — Ampere GPU and all TPU

bfloat16 uses 8 exponent bits (vs 5 in float16), giving it the same dynamic range as float32 (exponent range 2^{-12} to 2^{12}). No underflow risk → no GradScaler needed. Both notebooks use bfloat16 on TPU natively; the GPU v4.0 uses bfloat16 on Ampere+ GPUs and falls back to float16+GradScaler otherwise.

18. Batch Normalisation & Recalibration

BatchNorm forward pass (training mode)

```
mu_B = (1/B) * sum_i x_i batch mean sigma_B = sqrt( (1/B)*sum_i
(x_i-mu_B)^2 + eps ) batch std x_hat_i = (x_i - mu_B) / sigma_B y_i = gamma
* x_hat_i + beta learnable scale+shift
```

Running statistics (inference mode)

```
mu_run <- (1-momentum) * mu_run + momentum * mu_B sigma_run <- (1-momentum)
* sigma_run + momentum * sigma_B At inference: use mu_run, sigma_run
instead of batch statistics
```

BN Recalibration after SWA/EMA

The EMA/SWA model has averaged weights but stale running statistics (collected from the raw model, not the averaged one). Recalibration resets and re-estimates them using 25 training batches:

```
For each BN layer: reset mu_run = 0, sigma_run = 1, num_batches_tracked = 0
set momentum = None -> cumulative moving average mode: momentum = 1 /
num_batches_tracked Run 25 forward passes (no backward) through training
data -> mu_run, sigma_run now reflect EMA model's actual distributions
```

19. Stochastic Weight Averaging (SWA)

SWA maintains a model whose weights are the running arithmetic mean of the model at each epoch after epoch `swa_start = 30`:

```
theta_SWA <- theta_SWA + (theta_epoch - theta_SWA) / n_averaged which is  
equivalent to: theta_SWA = (1/n) * sum_{k=1}^n theta_{swa_start + k}
```

SWA differs from EMA: SWA weights all epochs equally while EMA exponentially discounts older epochs. SWA is known to converge to flatter minima of the loss surface, which generalise better.

20. Platt Scaling (Calibration)

Neural networks are often overconfident — outputting probabilities near 0 or 1 when the true uncertainty is much higher. Platt Scaling fits a logistic regression on the raw model outputs (logits or probabilities) against validation labels:

```
calibrated_prob = sigma( A * raw_prob + B ) Fitted by minimising: sum_i  
BCE( sigma(A * p_i + B) , y_i ) over validation set using sklearn  
LogisticRegression: cal.fit( probs.reshape(-1,1) , labels ) A =  
cal.coef_[0, 0] B = cal.intercept_[0]
```

After calibration, if the model outputs 0.9 for a set of images, approximately 90% of those images are truly defective. Before calibration the model may output 0.95+ on most defectives but only 80% are truly positive.

21. Evaluation Metrics

All metrics are derived from the confusion matrix: TP (true positives), TN (true negatives), FP (false positives), FN (false negatives).

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$ Precision = $TP / (TP + FP)$ 'of predicted defects, how many are real?' Recall = $TP / (TP + FN)$ 'of all real defects, how many did we catch?' F1 Score = $2 * Precision * Recall / (Precision + Recall) = 2*TP / (2*TP + FP + FN)$ 'harmonic mean' AUC-ROC = $P(\text{score(defective)} > \text{score(clean)})$ [ranking quality] Numerically: area under the curve plotting TPR vs FPR as the classification threshold sweeps 0 -> 1.

F1 is the primary metric because it balances precision and recall and is insensitive to class imbalance (unlike accuracy). AUC is threshold-independent and measures ranking quality.

22. Threshold Sweep — Vectorised F1 Maximisation

Instead of looping through 91 thresholds sequentially (slow), the notebooks use GPU broadcasting to evaluate all thresholds simultaneously:

```
probs shape: (N,) N = number of validation samples thrs shape: (91,)
linspace(0.05, 0.95, step=0.01) preds_all = (probs.unsqueeze(0) >=
thrs.unsqueeze(1)) shape: (91, N) # Each row is the prediction vector at
one threshold TP = (preds_all * labels).sum(1) shape: (91,) FP = (preds_all
* (1-labels)).sum(1) FN = ((1-preds_all) * labels).sum(1) P = TP /
clip(TP+FP, min=1) R = TP / clip(TP+FN, min=1) F1 = 2*P*R / clip(P+R,
min=1e-8) best_threshold = thrs[ argmax(F1) ]
```

23. Cascade Inference Decision Rule

The two-stage cascade combines Stage 1 (B0) and Stage 2 (Ensemble) via a conditional override:

```
p_B0 = sigma( B0(full_image) ) p_ensemble = mean_k( sigma( Ensemble_k(crop)
) ) over k folds p_final = p_B0 if p_B0 >= tau_1 p_ensemble otherwise tau_1
= 0.70 (Stage 1 confidence threshold) y_hat = 1 if p_final >= tau_blend,
else 0 tau_blend = mean of per-fold optimal thresholds (or loaded from
file)
```

The cascade is not a weighted blend — it is a strict conditional. If Stage 1 is highly confident (≥ 0.70), its probability is used directly. Only if Stage 1 is unsure does the expensive ensemble run.

24. Rim Cropping — HoughCircles

The Hough Circle Transform detects circles in an edge image by accumulating votes in a 3D parameter space (cx, cy, r).

```
For each edge pixel (x, y) at angle theta: a = x - r * cos(theta) b = y - r * sin(theta) Accumulator[a, b, r] += 1 Circle centre and radius: (a*, b*, r*) = argmax(Accumulator) Crop box: side = 2 * r * (1 + margin) margin = 0.10 x1 = max(0, cx - side/2) x2 = min(W, cx + side/2) crop resized to (out_size, out_size) via bilinear interpolation
```

Pre-processing: Gaussian blur with kernel (9,9) sigma=2 removes glare before Canny edge detection. Param1=100 is the upper Canny threshold; param2=30 is the accumulator threshold.

25. Grad-CAM (v4.0 GPU Only)

Grad-CAM generates a spatial heatmap showing which image regions most influenced the model's defect prediction. It uses gradients of the class score with respect to the final convolutional feature map.

```
Let A^k be the k-th channel of the last conv feature map shape: (H', W')
Let y be the binary class logit (before sigmoid) alpha_k = (1 / H'*W') *
sum_{i,j} dL/dA^k_{i,j} 'GAP of gradients' CAM = ReLU( sum_k alpha_k * A^k
) shape: (H', W') CAM_norm = (CAM - min(CAM)) / (max(CAM) - min(CAM) +
1e-8) Resize CAM_norm to full image size (W, H) via bilinear interpolation.
Hotspot: (cy, cx) = argmax(CAM_norm) Crop: half = min(H,W) * (0.5 +
pad_frac) pad_frac=0.15 patch = img[ cy-half : cy+half, cx-half : cx+half ]
```

ReLU is applied before the weighted sum to keep only the feature channels that have a positive influence on the defect prediction. Negative channels (that argue against a defect) are suppressed.

26. Pseudo-Labeling Confidence Filter

After training all k folds, each test image receives k probability estimates. Only images where all models strongly agree are kept as pseudo-labels for retraining:

```
p_i = (1/K) * sum_{k=1}^K p_{i,k} mean ensemble probability Keep image i
as pseudo-label if: p_i >= 0.95 -> pseudo-label y=1 (defective) p_i <= 0.05
-> pseudo-label y=0 (clean) Minimum required: 100 pseudo-labels for
retraining to proceed.
```

The 0.95/0.05 thresholds ensure that only images where the ensemble is overwhelmingly confident contribute to pseudo-labels. Ambiguous images (probabilities 0.1–0.9) are discarded to avoid teaching the model on uncertain guesses.

27. Vignette, Ring & Specular Highlight — Full Geometry

Vignette factor at pixel (x, y)

```
d(x,y) = sqrt( ((x - W/2)/(W/2))^2 + ((y - H/2)/(H/2))^2 ) Note: d is
normalised so the image corners have d = sqrt(2) ~ 1.414 and the image
edges have d = 1 v(x,y) = 1 - clip( d(x,y) * s , 0 , 0.6 ) s ~ Uniform(0.3,
0.7) pixel_out = pixel_in * v(x,y) Maximum darkening at corners: 1 - 0.6 =
0.4 (40% of original brightness)
```

Annular ring mask

```
d(x,y) = sqrt( (x - W/2)^2 + (y - H/2)^2 ) [pixels, not normalised]
ring(x,y) = { 1 if r1 <= d(x,y) <= r2 { 0 otherwise r1 ~ Uniform(0.2, 0.4)
* min(W,H) r2 = r1 + Uniform(5, 20) pixels pixel_out = clip( pixel_in +
ring(x,y) * tint * 30 , 0, 255 ) tint[c] ~ Uniform(0.8, 1.2) per channel
```

Specular highlight 2D Gaussian

```
mask(x,y) = exp( -[ ((x-cx)/rx)^2 + ((y-cy)/ry)^2 ] ) cx ~ Uniform(W/4,
3W/4) cy ~ Uniform(H/4, 3H/4) rx ~ Uniform(W/10, W/4) ry ~ Uniform(H/10,
H/4) intensity ~ Uniform(0.2, 0.7) pixel_out[c] = clip( pixel_in[c] +
mask(x,y) * intensity * 255 , 0, 255 )
```